

Contents

Test Data Change Log — Layer 345 (Baldwin Re, Activity 40)	3
Original state (before any testing)	3
ILSTermsLossSetAssociation — original state	3
Change 1 — Fixed the loss-set association (data integrity fix, not a deductible test)	3
Change 2 — Franchise Deductible test (bit 1)	3
Change 3 — Corrected Franchise Deductible test at the value actually in effect	4
Change 4 — Reverse Franchise Deductible test (bit 2, combined with bit 1)	4
Change 5 — Inner Aggregate Deductible test (bit 4, combined with bits 1+2)	4
Change 6 — Outer Aggregate Deductible test (bit 8, tested ISOLATED — not combined)	5
Current state of ILSTermsID = 345 as of this log	5
ILSTermsLossSetAssociation current state	5
Decisions still pending (flagged, not yet made)	5
Suggested revert script (NOT yet run — for reference when ready)	6
Phase 2: Reinstatement Premium Waterfall Testing (in progress)	6
Change 7 — Check whether real ReinstatementPremium data exists on 345/346	6
Change 8 — Important correction to the draft test row before running it	7
Change 9 — Reinstatement Premium test row created	8
Change 10 — Synthetic data to test the fully-exhausted ELSE branch	8
Phase 3: CATBOND Unlimited-Occurrence-Limit Carve-Out (item 6)	9
Change 11 — Confirmed layer 344's real terms trigger this branch	9
Change 12 — Unexpected discovery: an orphaned second CATBOND layer (ILSTermsID = 341)	9
Change 13 — Isolated test layer cloning 344's real terms	10
Change 14 — Result: FINDING CONFIRMED	10
Phase 4: RPP Sublayer (item 7)	11
Change 16 — New isolated test instrument and activity	11
Change 17 — Two-layer RPP structure	11
Change 18 — Unexpected extra loss-set association on layer B (data anomaly, not a code bug)	11
Change 19 — Result after removing the stray association: [CONFIRMED WORKING]	12
Change 20 — Stage 2, not yet run: a specific, already-identified risk	12
Change 21 — Stage 2 executed: FINDING CONFIRMED	12
Change 22 — Proposed fix for the @ROL/RPP issue, drafted and mechanically tested	13
Phase 5: AGGTRIGGER — Annual ILW Binary Payout (item 8)	14
Change 23 — Confirmed no real AGGTRIGGER layer exists	14
Change 24 — Setup mix-up, caught and corrected before it affected anything	14
Change 25 — Result: [CONFIRMED WORKING]	15
Phase 6: OCCTRIGGER — Per-Occurrence Nth-Event Binary Payout (item 9)	15
Change 26 — Confirmed no real OCCTRIGGER layer exists; isolated test built	15
Change 27 — Result: [CONFIRMED WORKING], strong single-data-point proof	15
Phase 7: PRORATA — Ratio-Scaled Notional Tranche (item 10)	16
Change 28 — Market context checked first, then code traced against it	16
Change 29 — Confirmed no real PRORATA layer exists; isolated test built specifically to trigger the predicted bug	16
Change 30 — Result: [CONFIRMED FINDING], unambiguous	16
Change 31 — Fix drafted, applied, and confirmed working (with a real deployment detour along the way)	17
Phase 8: OCCILW — Occurrence-Capped, Proportionally-Scaled ILW (item 11)	18
Change 32 — Confirmed no real OCCILW layer exists; code check found no PRORATA-style bug	18

Change 33 — Result: [CONFIRMED WORKING]	18
Phase 9: ELSE Catch-All — Investigation Redirected to a Real Finding (item 14)	19
Change 34 — Pre-test check found something more significant than expected	19
Change 35 — Systematically checked how GU layers actually receive loss data	19
Change 36 — Confirmed directly on real, already-completed production data	19
[CORRECTED] Structural pattern confirmed — but NOT confirmed as a production data gap	20
Change 37 - Synthetic test confirming the GU/CATBOND mechanism works correctly with real loss data	20
Change 15 — Proposed fix drafted (IMPORTANT: unvalidated against product intent — see caveat below)	21
Phase 10: Drop-Down Cover (item 15)	22
Change 38 - Verified drafted fix against PDF's own worked example before deploying . . .	22
Change 39 - Confirmed StepLayerSequenceNumber has never been wired end-to-end . . .	22
Change 40 - Engine-side wiring built and deployed	23
Change 41 - Test setup, with two real mistakes caught and fixed along the way	23
Change 42 - Result: [CONFIRMED WORKING], exact match to the PDF's documented example	23
Phase 11: Multi-Child Sourcing/Inuring at One Parent (structural question, beyond original item list)	24
Change 43 - Question raised: does a parent with MULTIPLE children correctly combine them, with sourced vs. inuring applied independently?	24
Change 44 - Test setup: one CATBOND parent, three CATXOL children with distinct behaviors	24
Change 45 - Result: [CONFIRMED WORKING]	25
Phase 12: Coinsurance (item 13) - Direction Confirmed, Fix Built and Tested	25
Change 46 - Direction confirmed by the person running this investigation	25
Change 47 - Fix implemented	25
Change 48 - Test setup and result: [CONFIRMED WORKING]	26
Phase 13: Coinsurance - NetLoss_xRIP Extension and a Serious Deployment Mistake Caught	26
Change 49 - Decision: extend the fix to NetLoss_xRIP	26
Change 50 - Deployment mistake: the actual EXECUTE line was deleted, not just edited . .	26
Change 51 - Result: [CONFIRMED WORKING]	27
Phase 14: Quota Share Ceded % - Full Multi-Dimensional Implementation (item 12)	27
Change 52 - Two blocking questions resolved directly by the person running this investigation	27
Change 53 - Design evolution: from a flat column to a unified, multi-dimensional rules table	27
Change 54 - Peril dimension added after being raised separately	28
Change 55 - Engine-side implementation	28
Change 56 - Real-table protection: synthetic substitute used for testing	28
Change 57 - Test design and execution: 5 events, each isolating one specificity level . . .	29
Change 58 - Priority correction confirmed and applied	29
[CONFIRMED WORKING - mechanism] / [PENDING - final priority reverification] Summary for item 12	29
Phase 15: Quota Share Ceded % - Design Redesigned Using Real AIR Geography Data (item 12, final version)	30
Change 59 - Real ARAPL_populate_ILSTerms source finally obtained and reviewed	30
Change 60 - Real AIR geography hierarchy discovered and confirmed usable	30
Change 61 - Design iterated substantially before settling	30
Change 62 - Final lookup design (four-level priority, two tables)	31
Change 63 - Test setup and execution: 5 events covering all 4 priority levels plus genuine fallback	31
Change 64 - Test-table join reverted to the real table, confirmed via full procedure re-read .	32

Test Data Change Log — Layer 345 (Baldwin Re, Activity 40)

This documents every modification made to real production data during deductible-mechanism testing, in the order they happened, so the original state can be restored and the test history is traceable.

Layer being modified throughout: ILTermsID = 345 (one of two CATXOL layers under ILSInstrumentID = 40, "Baldwin Re Ltd. Series 2025-1 Class A", ParentILTermsID = 344)

Original state (before any testing)

Field	Original value
DeductibleType	NULL
CompanyLossMinimum	NULL
CompanyLossMaximum	NULL
AggregateDeductible	NULL

ILTermsLossSetAssociation — original state

ILTermsID	LossSetID	LMF
345	1	75000000

This was wrong — LossSetID = 1 pointed at placeholder/leftover test data (LSActivityID = 1), not real data for Activity 40. Also, LMF = 75000000 was clearly invalid (a Loss Modification Factor should be ~1.0, not 75 million).

Change 1 — Fixed the loss-set association (data integrity fix, not a deductible test)

```
UPDATE ARAPL_Configuration_v13.dbo.ILTermsLossSetAssociation
SET LossSetID = 900040,    -- correct loss set, LSActivityID = 40 (matches Activity)
    LMF = 1.0
WHERE ILTermsID = 345
```

Status: this is a genuine correction, not a test artifact. This should almost certainly remain in place — layer 345 had no real loss data flowing through it at all until this was fixed. (Note: layers 344 and 346 were checked and found to have **no loss-set association at all** — this may still need addressing separately, outside the scope of this log.)

Change 2 — Franchise Deductible test (bit 1)

```
UPDATE ARAPL_Configuration_v13.dbo.ILTerms
SET DeductibleType = 1,
    CompanyLossMinimum = 450000000    -- initial test value
WHERE ILTermsID = 345
```

Result: initial test at \$450M threshold — confirmed 137→133 events... (Note: this initial \$450M test was run before we discovered CompanyLossMinimum had separately drifted to \$75,000,000 by the time of the actual confirmed test below — the \$450M value shown here was superseded.)

Change 3 — Corrected Franchise Deductible test at the value actually in effect

At the point of the confirmed test, CompanyLossMinimum was found to be 750000000 (not \$450M — cause of this discrepancy was not fully traced, flagged at the time as worth double-checking separately).

Confirmed test result at this value:

```
-- State at time of confirmed test:
-- DeductibleType = 1, CompanyLossMinimum = 750000000
```

- EventsBelow75M (raw data check) = 4
- Debug_GRYELT (before filter) = 137 rows
- Debug_GRYELT_PostDed (after filter) = 133 rows
- **Result: 137 – 4 = 133 [CONFIRMED WORKING]**

Change 4 — Reverse Franchise Deductible test (bit 2, combined with bit 1)

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET DeductibleType = 3,          -- 1 (Franchise) + 2 (Reverse Franchise)
    CompanyLossMaximum = 900000000
WHERE ILSTermsID = 345
-- CompanyLossMinimum remained 750000000
```

First attempt — failed. SurvivingMax came back as 1,125,000,000, above the \$900M ceiling — proved the deployed procedure was missing the Reverse Franchise check entirely in the shared pre-filter block (a gap in the fix I'd given earlier, not a data issue).

Procedure was corrected (the IF @isFranchiseDeductible = 1 ... ELSE ... block was replaced with a combined IF @isFranchiseDeductible = 1 OR @isReverseFranchiseDeductible = 1 ... WHERE (...) AND (...) version — see the procedure's own change history for the exact code).

Re-tested after the code fix, same DeductibleType = 3 / CompanyLossMinimum = 750000000 / CompanyLossMaximum = 900000000: - SurvivingMin = 86,576,910 (≥ \$75M [OK]) - SurvivingMax = 873,926,400 (≤ \$900M [OK]) - **Result: [CONFIRMED WORKING]** (both floor and ceiling enforced together)

Change 5 — Inner Aggregate Deductible test (bit 4, combined with bits 1+2)

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET DeductibleType = 7,          -- 1+2+4 (Franchise + Reverse Franchise + Inner Aggregate)
    AggregateDeductible = 500000000
WHERE ILSTermsID = 345
-- CompanyLossMinimum = 750000000, CompanyLossMaximum = 900000000 (unchanged from Change 4)
```

Result: [CONFIRMED WORKING]. Checked via Debug_ReinstAggYLT.NetNetLoss against tmp-NetLoss – 50,000,000 (floored at 0) across all 40 non-zero years — every row matched exactly, including specific hand-verified years: - Year 881: 45,049,800 → 0 - Year 4181: 61,973,400 → 11,973,400 - Year 7243: 300,000,000 → 250,000,000

This test also confirmed all three bits (1+2+4) combine correctly on one layer simultaneously.

Change 6 — Outer Aggregate Deductible test (bit 8, tested ISOLATED — not combined)

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET DeductibleType = 8,           -- Outer Aggregate ONLY
    AggregateDeductible = 500000000 -- different value/purpose than Change 5's $50M
WHERE ILSTermsID = 345
-- CompanyLossMinimum, CompanyLossMaximum: values retained in the table but NOT ACTIVE,
-- since DeductibleType=8 alone means bits 1/2 are OFF regardless of stored values
```

Important finding confirmed by this test: Outer Aggregate Deductible (bit 8) and Franchise/Reverse Franchise (bits 1/2) are **mutually exclusive** in the current code — the branch structure is IF @isOuterAggregateDeductible = 1 ... ELSE (franchise checks) ..., so setting bit 8 silently disables bits 1/2 even if their threshold values are still stored on the row. This was tested deliberately in isolation for this reason.

Result: [CONFIRMED WORKING]. - Year 9907 (raw annual total \$33,197,860, below \$500M) → **absent entirely** from Debug_GRYELT_PostDed [OK] - Year 434 (raw annual total \$1,125,000,000, single event) → scaled to exactly 625,000,000 = 1,125,000,000 × (625M/1,125M) [OK] - Year 7243 (two real events) → correctly split proportionally into two separate scaled rows summing to the same ratio [OK] (confirms scaling is applied per-event, not just per-year lump sum)

Current state of ILSTermsID = 345 as of this log

Field	Current value	From which change
DeductibleType	8	Change 6 (most recent)
CompanyLossMinimum	750000000	Change 3 (inactive while DeductibleType=8)
CompanyLossMaximum	9000000000	Change 4 (inactive while DeductibleType=8)
AggregateDeductible	5000000000	Change 6 (overwrote Change 5's 500000000)

ILSTermsLossSetAssociation current state

ILSTermsID	LossSetID	LMF
345	900040	1.0

Decisions still pending (flagged, not yet made)

1. **Should the ILSTermsLossSetAssociation fix (Change 1) be kept permanently?** This looks like a genuine data-quality correction rather than a test artifact — recommend keeping it, but confirm with whoever owns this data before treating it as final.
2. **Should layers 344 and 346 also get loss-set associations added?** Discovered during Change 1's investigation — outside this log's scope but noted here so it isn't lost.

3. **Should DeductibleType/CompanyLossMinimum/CompanyLossMaximum/AggregateDeductible be reverted to NULL on layer 345 now that all four mechanisms are proven?** Recommended, since these are test values with no business meaning for the real Baldwin Re bond — but not yet done, pending confirmation this log is sufficient to resume testing later if needed.
4. **The \$450M vs. \$75M Franchise threshold discrepancy (Change 2 vs. Change 3)** was never fully explained — flagged at the time, not resolved. Worth a quick investigation if anyone revisits this data.

Suggested revert script (NOT yet run — for reference when ready)

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET DeductibleType = NULL,
    CompanyLossMinimum = NULL,
    CompanyLossMaximum = NULL,
    AggregateDeductible = NULL
WHERE ILSTermsID = 345
```

```
-- Decide separately whether to revert the ILSTermsLossSetAssociation fix (Change 1) -
-- recommend NOT reverting this one, as it corrects a genuine data error.
```

Phase 2: Reinstatement Premium Waterfall Testing (in progress)

Scope note: unlike Phase 1 (deductible-type mechanisms, all tested on layer 345), this phase tests the CATXOL/RPP/CATBOND branch’s banded reinstatement premium calculation — a completely different piece of logic (the WHILE @ReinstID <= @ReinstMAXID dynamic-SQL loop that builds a tiered CASE expression). Layer 345 is currently left in its Phase 1 end-state (DeductibleType = 8, AggregateDeductible = 5000000000 — see “Current state” above) and has **not** been touched for this phase yet.

Change 7 — Check whether real ReinstatementPremium data exists on 345/346

```
SELECT ILSTermsID, ReinsuranceTypeCode, ReinstatementPremium, Reinstatement,
       ReinstProvisionalAmount, ReinstProvisionalTime,
       PercentAnnualSpread, Premium, PrincipalAmount
FROM ARAPL_Configuration_v13.dbo.ILSTerms
WHERE ILSTermsID IN (345, 346)
```

Status: CONFIRMED — both empty, as expected.

ILSTermsID	ReinsuranceTypeCode	ReinstatementPremium	Reinstatement	ReinstProvisionalAmount	ReinstProvisionalTime	PercentAnnualSpread	Premium	PrincipalAmount
345	CATXOL	NULL	0	NULL	NULL	NULL	NULL	NULL
346	CATXOL	NULL	1	NULL	NULL	NULL	NULL	NULL

Neither layer has a usable ReinstatementPremium string — confirms a dedicated test row is needed. Note in passing: Reinstatement (the plain integer count field) is populated differently on each (0 vs 1), but this is irrelevant to the branch’s actual reinstatement-premium logic, which is driven entirely by parsing the ReinstatementPremium string, not this integer field — another small confirmation of something already suspected about this code.

Change 8 — Important correction to the draft test row before running it

Caught before execution: the original draft used `InuringSequenceNumber = 20` to keep the row “isolated.” This would have silently broken the test — `@ROL` (rate-on-line, the multiplier every reinstatement band’s premium is based on) only resolves to a real value when `InuringSequenceNumber = 1`:

```
@ROL = case when InuringSequenceNumber = 1 or (@isRPP = 1 and InuringSequenceNumber = 2)
            then ISNULL(PercentAnnualSpread, Premium / PrincipalAmount)
            else 0 end
```

At `InuringSequenceNumber = 20`, `@ROL` would resolve to 0, and every reinstatement premium in the banded formula would compute as 0 regardless of the configured rates — the test would run without erroring, but would prove nothing.

Corrected: `InuringSequenceNumber = 1` instead of 20. This does not conflict with layer 344 (the real parent, also `InuringSequenceNumber = 1`) — this field is evaluated fresh per-row, per-loop-iteration, not as a table-wide unique key, so two independent rows both carrying 1 is safe. Isolation from the real Baldwin Re chain is instead achieved via `ParentILSTermsID = 0 / ParentRelationType = 1` (sourced/top layer — nothing points to this row as a child, and it doesn’t subtract from or get subtracted by anything real).

Final corrected insert (not yet run):

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSTerms
(
  ILSInstrumentID, AnalysisSID, ReinsuranceTypeCode, ILSType,
  CurrencyCode, InceptionDate, ExpirationDate,
  InuringSequenceNumber, ParentILSTermsID, ParentRelationType,
  OccurrenceLimit, OccurrenceRetention,
  AggregateLimit, AggregateRetention,
  ReinstatementPremium,
  PercentAnnualSpread, PrincipalAmount,
  ParticipationGross, ParticipationNet,
  IsDeleted, DeductibleType)
VALUES
(
  40, 40, 'CATXOL', 'CAT Bond',
  'USD', '2025-07-01', '2029-06-30',
  1, 0, 1,
  500000000, 1000000000,
  NULL, NULL,
  '1@50,1@100',
  0.05, 500000000,
  1, 1,
  'N', NULL)
-- InuringSequenceNumber=1 (fixed),
-- isolated via Parent fields instead
```

Then link to real loss data:

```
SELECT ILSTermsID FROM ARAPL_Configuration_v13.dbo.ILSTerms
WHERE ILSInstrumentID = 40 AND ReinstatementPremium = '1@50,1@100'
```

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation
(
  ILSTermsID, LossSetID, LMF)
VALUES (<new ILSTermsID>, 900040, 1)
```

Not yet executed — next step is to run this, then check the post-occurrence-limited annual distribution specific to THIS layer’s own \$50M/\$100M terms (its own version of the exercise done earlier for layer 345’s \$300M/\$500M terms), since the two layers’ terms are different enough that 345’s known distribution doesn’t transfer.

Change 9 — Reinstatement Premium test row created

New, dedicated, isolated ILTermsID = 2805 created (not a further edit to 345/346), per Change 8's corrected insert. Linked to real loss data (LossSetID = 900040, same as layer 345) via ILSTermsLossSetAssociation.

Layer 2805 terms: OccurrenceLimit = \$50M, OccurrenceRetention = \$100M, ReinstatementPremium = '1@50,1@100' (2 reinstatements: 50% rate, then 100% rate), PercentAnnualSpread = 0.05, PrincipalAmount = \$50M. Resolves to AggregateLimit = (2+1) × \$50M = \$150M.

Result against real data (140 years from YELT_40, LossSetID 900040): [CONFIRMED WORKING] across all three regions of the banded formula, verified against every single year with zero discrepancies:

Region	Example year(s)	tmpNetLoss	NetNetLoss	ReinstatementPremium	Formula check
Zero-loss	9907, 9646, 7781, 6153, 5358, 1580, 4170	0	0	0	—
Band 1, partial	125	9,600,176.25	same	240,004.41	× 0.025 [OK]
Band 1, partial	8845	11,777,100	same	294,427.50	× 0.025 [OK]
Band 1, partial	4687	29,476,700	same	736,917.50	× 0.025 [OK]
Band 1, full	~133 years at exactly \$50M	50,000,000	50,000,000	1,250,000	50M × 0.025 [OK]
Band 2, full	7243	100,000,000	100,000,000	3,750,000	1.25M + (50M × 0.05) [OK]

Real data could not reach beyond \$100M for this layer's terms — the fully-exhausted ELSE branch (flat premium ceiling, layer capacity fully consumed) needed synthetic data. See Change 10.

Change 10 — Synthetic data to test the fully-exhausted ELSE branch

First attempt — instructive failure, not a bug. Built a synthetic loss set (LossSetMetaData, ActivityId = 999001) with **one** event at PreCatLoss = 500,000,000:

```
INSERT INTO ARAPL_Configuration_v13.dbo.LossSetMetaData
(ActivityId, LossSetName, isDeleted, isModeledLoss)
VALUES (999001, 'SYNTHETIC TEST - Reinstatement Premium ceiling check', 0, 'Y')
-- generated LossSetID = 900042 (a duplicate row, LossSetID = 900043, was accidentally
-- created by a double-run and deleted immediately)
```

Result: ReinstatementPremium = 1,250,000 (only Band 1) — **not a bug**. Occurrence terms cap *each event individually* at OccurrenceLimit = \$50M before the aggregate step ever sees the total. A single event, however large, can never contribute more than \$50M to the year — so one event alone can never push the annual total past the first \$50M boundary, regardless of its raw size. The test needed the wrong shape of data (many events in one year, not one enormous event).

Corrected synthetic data — three events in the same year, each occurrence-limited to \$50M, summing to \$150M:

```
DROP TABLE IF EXISTS ARAPL_Data_v13.dbo.YELT_999001
SELECT * INTO ARAPL_Data_v13.dbo.YELT_999001
FROM (VALUES
(CAST(1 AS BIGINT), CAST(1 AS INT), CAST(1 AS INT), CAST(500000000 AS FLOAT)),
(CAST(2 AS BIGINT), CAST(1 AS INT), CAST(2 AS INT), CAST(500000000 AS FLOAT)),
(CAST(3 AS BIGINT), CAST(1 AS INT), CAST(3 AS INT), CAST(500000000 AS FLOAT))
) AS t(EventID, YearID, DayID, PreCatLoss)
```

Layer 2805 temporarily repointed to this synthetic loss set:

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation
SET LossSetID = 900042, LMF = 1
WHERE ILSTermsID = 2805
```

Result: [CONFIRMED WORKING]. tmpNetLoss = 150,000,000, NetNetLoss = 150,000,000 (fully consumes AggregateLimit), ReinstatementPremium = 3,750,000 (the flat ceiling — the formula's final ELSE branch, confirmed for the first time).

Reinstatement Premium waterfall (Phase 2 goal) is now fully tested — all three regions (Band 1 partial/full, Band 2, fully-exhausted ceiling) confirmed working, using both real data and a targeted synthetic case for the one region real data couldn't reach.

Open items from Phase 2, not yet actioned

- Layer 2805 is currently pointed at the **synthetic** loss set (LossSetID = 900042), not the real one (900040) — revert this if 2805 is needed for anything further with real data.
- The synthetic LossSetMetaData/YELT_999001 objects (ActivityId = 999001) still exist — decide whether to keep them for future re-tests of this specific branch, or clean them up.
- Layer 2805 itself is a test-only row under real ILSInstrumentID = 40 — decide whether to soft-delete it (IsDeleted = 'Y') once Phase 2 is fully closed out, so it doesn't linger indefinitely alongside real Baldwin Re layers.

Phase 3: CATBOND Unlimited-Occurrence-Limit Carve-Out (item 6)

Change 11 — Confirmed layer 344's real terms trigger this branch

```
SELECT ILSTermsID, ReinsuranceTypeCode, OccurrenceLimit, OccurrenceRetention,
       AggregateLimit, AggregateRetention, PrincipalAmount, PercentAnnualSpread
FROM ARAPL_Configuration_v13.dbo.ILSTerms WHERE ILSTermsID = 344
```

Result: OccurrenceLimit = NULL (→ resolves to @DEFAULTLIMIT = 1e15), OccurrenceRetention = 5000000000, AggregateLimit = NULL (→ also resolves to 1e15, since ReinsuranceTypeCode ≠ 'OCCILW'), PrincipalAmount = 2500000000, PercentAnnualSpread = 0.0375.

Confirms layer 344 genuinely runs through the code's unlimited-occurrence-limit branch in production today (IF @ReinsuranceTypeCode = 'CATBOND' AND @OccurrenceLimit = @DEFAULTLIMIT), which explicitly skips reinstatement-based AggregateLimit expansion and honors OccurrenceRetention/AggregateRetention/AggregateLimit exactly as resolved — with **no ceiling derived from PrincipalAmount anywhere in this branch.**

Change 12 — Unexpected discovery: an orphaned second CATBOND layer (ILSTermsID = 341)

While searching for existing CATBOND layers, found ILSTermsID = 341 — same ILSInstrumentID = 40, terms nearly identical to 344 (OccurrenceRetention = 5000000000, PrincipalAmount = 2500000000, PercentAnnualSpread = 0.0375), but **AnalysisSID = NULL** and **no loss-set association.**

Confirmed harmless to existing tests: the main loop filters WHERE AnalysisSID = @ActivityID, and NULL = 40 is never true in SQL Server — layer 341 has never been pulled into any run this session, for Activity 40 or otherwise. Flagged as an orphaned/incomplete row worth mentioning to whoever owns this data, but not something blocking this testing.

Change 13 — Isolated test layer cloning 344's real terms

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSTerms
(ILInstrumentID, AnalysisSID, ReinsuranceTypeCode,
CurrencyCode, InceptionDate, ExpirationDate,
InuringSequenceNumber, ParentILSTermsID, ParentRelationType,
OccurrenceLimit, OccurrenceRetention,
AggregateLimit, AggregateRetention,
PrincipalAmount, PercentAnnualSpread, Reinstatement,
ParticipationGross, ParticipationNet,
IsDeleted, DeductibleType)
VALUES
(40, 40, 'CATBOND',
'USD', '2025-07-01', '2029-06-30',
1, 0, 1,
NULL, 5000000000,
NULL, NULL,
2500000000, 0.0375, 0,
1, 1,
'N', NULL)
```

Created as ILSTermsID = 2806, isolated (ParentILSTermsID = 0, ParentRelationType = 1) so it sources directly on real loss data rather than through the 344/345/346 inuring chain (which currently carries leftover Phase 1/2 test state on 345, and no loss data at all on 346).

Linked to real data:

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation (ILSTermsID, LossSetID, LM
VALUES (2806, 900040, 1)
```

Change 14 — Result: FINDING CONFIRMED

```
SELECT TOP 10 EventID, YearID, Loss
FROM ARAPL_Analysis_V13.dbo.Debug_OccYELT
WHERE ActivityID = 40 AND ILSTermsID = 2806
ORDER BY Loss DESC
```

Result: the top 10 (and, per the earlier annual-distribution check, at least ~78 of the 140 simulated years) all show Loss = 625,000,000 — exactly 1,125,000,000 (the real modeled event) – 500,000,000 (OccurrenceRetention), with no ceiling applied.

[CONFIRMED FINDING]: this branch allows a modeled loss (\$625,000,000) to exceed the bond's own actual PrincipalAmount (\$250,000,000) by 2.5x. For a fully-collateralized CAT bond, this is not physically possible — the instrument cannot lose more than its own posted principal. Because more than half the years in this catalog hit the same dominant event, this isn't a rare edge case; it would affect the majority of the simulated results for this instrument if run through this branch as currently written.

Root cause: the branch's own code comment states it deliberately skips the reinstatement-based AggregateLimit expansion for the unlimited-occurrence-limit case, "honoring OccurrenceRetention/AggregateRetention/AggregateLimit as-is" — but none of those three fields, nor anything else in this branch, ever caps the result against PrincipalAmount. The cap that should exist (a CAT bond can never lose more than its own principal) is simply absent from this code path.

Open items from Phase 3, not yet actioned

- Test layer 2806 still exists, linked to real loss data — decide whether to soft-delete once this finding is fully documented/handed off.

- Layer 341 (orphaned, AnalysisSID = NULL) was discovered but not investigated further or resolved — flagged for whoever owns this data.
 - **This finding has not yet been proposed as a fix** — only confirmed as a gap. A fix would need to add LEAST(computed_loss, PrincipalAmount)-style capping specifically within the unlimited-occurrence-limit branch, without disturbing the standard (finite OccurrenceLimit) CATXOL/CATBOND path already confirmed working in Phase 1/2.
-

Phase 4: RPP Sublayer (item 7)

Risk flagged before setup: @isRPP is computed once, instrument-wide (not per-row), so this test could not be built on the shared Baldwin Re instrument (ILSInstrumentID = 40) without risking silent corruption of every already-confirmed result on that instrument. Built on a genuinely new, isolated instrument instead.

Change 16 — New isolated test instrument and activity

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSData (ILSName, Security_or_Reinsurance_Type)
VALUES ('SYNTHETIC TEST - RPP structure', 'CAT Bond')
-- generated ILSInstrumentID (noted as @NewInstrumentID)
```

```
INSERT INTO ARAPL_Configuration_v13.dbo.ILSActivityMonitor
(JobId, UserId, ARADbName, AcctName, AcctGrpId,
AnalysisType, ShapeId, Peril, JobStatus, CurrencyCode)
VALUES (1000, 'TestUser', '', '', <@NewInstrumentID>, 0, 0, 0, 0, 'USD')
-- generated ActivityID = 338 (both ActivityID and LossSetID/ILSInstrumentID confirmed identical)
-- columns during this phase - could not be set explicitly, had to capture generated values)
```

Note: before building on ActivityID/ILSInstrumentID = 1 as originally suggested, checked and found it already in use by a real, separate, completed instrument — “CB-Aquila Re I Ltd. Series 2023-1 Class A-1 Reset 2025” (a different real CAT bond from Baldwin Re, with 3 real QS layers + 1 CATBOND layer). Avoided reusing it; used ActivityID = 338 (auto-generated) instead. Worth remembering this instrument exists — its QS layers will be directly useful for Phase 5/6 (items 8-12).

Change 17 — Two-layer RPP structure

Layer B (container) — ILSTermsID = 2807, InuringSequenceNumber = 1, ParentRelationType = 4 (the exact condition @isRPP’s existence check keys on), ParentILSTermsID = 0.

Layer A (RPP protection) — ILSTermsID = 2808, InuringSequenceNumber = 2, ParentILSTermsID = 2807, OccurrenceLimit = \$50M/OccurrenceRetention = \$100M, ReinstatementPremium = '1@50,1@100', PercentAnnualSpread = 0.05. **Stage 1: typed ReinsuranceTypeCode = 'CATXOL'** (deliberately not 'RPP' yet), to isolate the swap mechanism itself from a separate concern identified during design (see Change 20).

Both linked to real Baldwin Re loss data (LossSetID = 900040) for realistic event shapes.

Change 18 — Unexpected extra loss-set association on layer B (data anomaly, not a code bug)

First run produced a puzzling result: layer B’s final output showed ~\$100,000,000 per year — implausible, since B should only receive layer A’s already-swapped, premium-scale output (~\$1.25M) via the underlying-layers mechanism.

Investigated and found: ILSTermsLossSetAssociation had a row for ILSTermsID = 2807 (layer B) pointing at LossSetID = 900040 — **never intentionally created**. Neither of us had inserted

this. Root cause not identified (possibly a default/trigger on the table, or an artifact of how the row was created) — flagged as an open mystery, not resolved.

Fix:

```
DELETE FROM ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation WHERE ILSTermsID = 2807
```

This gave B a direct, unintended connection to raw real event data, completely bypassing the inuring/underlying-layers mechanism — explains the ~\$100M figures exactly (B's own OccLimit = \$100M applied directly to real ~\$1.125B events).

Change 19 — Result after removing the stray association: [CONFIRMED WORKING]

Re-ran; checked Debug_LayerTerms (confirmed @isRPP triggered correctly — layer 2808's ROL = 0.05 could only be nonzero via the @isRPP=1 AND InuringSequenceNumber=2 branch of the @ROL formula, since InuringSequenceNumber ≠ 1), then Debug_tmpGrLoss for layer 2807 (B) — the real proof, since this is B's *final* output after receiving whatever flowed up from A.

Result: B's output matches A's swapped premium values exactly, correctly propagated through the inuring chain: - Standard \$50M-loss years: \$1,250,000 on both A and B [OK] - Year 125: \$240,004.41 on both [OK] · Year 4687: \$736,917.50 on both [OK] · Year 8845: \$294,427.50 on both [OK] - **Year 7243** (two-event, fully-exhausted year): combined \$3,750,000 ceiling correctly splits into **two \$1,875,000 rows** on B, proportionally back-allocated per event — matching the same back-allocation behavior confirmed in Phase 2 on layer 2805 - All 7 zero-loss years correctly absent from both A and B

[CONFIRMED WORKING] — Stage 1 (RPP mechanism with a CATX0L-typed protection layer) is fully proven: both the swap itself and its correct propagation through the inuring chain.

Change 20 — Stage 2, not yet run: a specific, already-identified risk

Concern flagged during design, not yet tested: inside the CATX0L/RPP/CATBOND branch, there's an override specific to ReinsuranceTypeCode = 'RPP':

```
SET @ROL = CASE WHEN @ReinsuranceTypeCode = 'RPP' THEN @RPPROL ELSE @ROL END
```

@RPPROL only gets a nonzero value when InuringSequenceNumber = 1:

```
@RPPROL = case when InuringSequenceNumber = 1 then ISNULL(LayerPremium, 0) else 0 end
```

Layer A sits at InuringSequenceNumber = 2 — meaning if A is switched from CATX0L to RPP, this override would force @ROL to 0 (from @RPPROL's default), **clobbering the correctly-working ROL = 0.05 just confirmed in Change 19**, which came from the *general* formula's dedicated (@isRPP=1 AND InuringSequenceNumber=2) carve-out. This looks like it could be a real, separate bug specific to layers explicitly typed 'RPP' in this exact structural position — not yet confirmed.

Not yet executed. Next step: change layer 2808's ReinsuranceTypeCode from 'CATX0L' to 'RPP', everything else identical, re-run, and check whether ROL/ReinstatementPremium breaks as predicted.

Change 21 — Stage 2 executed: FINDING CONFIRMED

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET ReinsuranceTypeCode = 'RPP' WHERE ILSTermsID = 2808
```

Re-ran and checked Debug_ReinstAggYLT (captured *after* the branch-specific override runs, unlike the earlier Debug_LayerTerms check which was mistakenly used first and couldn't detect this — corrected mid-test).

Result: every one of the 140 years now shows `ReinstatementPremium = 0`, with `NetNetLoss/tmpNetLoss` completely unchanged from Stage 1's confirmed values (e.g. year 106 still 50,000,000, year 7243 still 100,000,000) — proving only the premium calculation broke, not the underlying loss computation.

[CONFIRMED FINDING]: for a layer genuinely typed `ReinsuranceTypeCode = 'RPP'`, sitting at the structurally-required `InuringSequenceNumber = 2` position (the only position an RPP protection layer can occupy, by the design of the `@isRPP` mechanism itself), the reinstatement premium calculation is unconditionally zero — regardless of any real rates configured.

Root cause: the branch-specific override `SET @ROL = CASE WHEN @ReinsuranceTypeCode = 'RPP' THEN @RPPROL ELSE @ROL END` replaces the correctly-computed general-formula value (which has a dedicated carve-out: `@isRPP=1 AND InuringSequenceNumber=2`) with `@RPPROL`, which is only ever nonzero when `InuringSequenceNumber = 1` — a position the RPP protection layer can never occupy by design (that position is reserved for the container layer, `ParentRelationType = 4`). **The override and the general formula's own carve-out directly contradict each other, and the override always wins**, since it runs later in the branch. This means: every fix and mechanism confirmed working in Stage 1 (the swap itself, correct inuring propagation, correct back-allocation) works perfectly for a CATXOL-typed layer occupying this position — but breaks the moment the layer is honestly typed 'RPP', which is presumably how this structure is meant to be used in practice.

Second confirmed finding for Phase 4 (alongside Change 14/15's CATBOND principal-cap issue in Phase 3) — this one has **no proposed fix yet**, and unlike the CATBOND finding, it isn't just "missing" logic — it's two pieces of *existing* code directly contradicting each other, with no obvious reason from the code alone which one reflects the actual intent. Needs product/business input on which value `@ROL` is *supposed* to take for a real RPP layer before any fix is drafted.

Open items from Phase 4, not yet actioned

- **Stray ILSTermsLossSetAssociation row on layer B (Change 18)** — root cause never identified. Worth investigating separately, as it could affect other real layers if it's a systemic default/trigger behavior, not an isolated fluke.
- Test instrument (`ILSInstrumentID` from Change 16), activity 338, and layers 2807/2808 all still exist — decide on cleanup/retention once this phase is fully handed off.

Change 22 — Proposed fix for the @ROL/RPP issue, drafted and mechanically tested

Fix: make the RPP protection layer (position 2) read `LayerPremium` from its container's row (position 1) instead of its own row, since the original formula's `InuringSequenceNumber = 1` condition can structurally never be true for the protection layer:

```
@RPPROL = CASE WHEN @isRPP = 1 AND InuringSequenceNumber = 2 THEN
    (SELECT ISNULL(LayerPremium, 0) FROM ARAPL_Configuration_v13.dbo.ILSTerms
     WHERE ILSInstrumentID = b.ILSInstrumentID AND AnalysisSID = b.AnalysisSID
     AND InuringSequenceNumber = 1 AND ParentRelationType = 4 AND IsDeleted = 'N')
ELSE 0 END
```

Test setup: populated the container layer's `LayerPremium = $2,000,000` (previously always NULL, part of why this was never exercised), applied the fix, re-ran.

Result: [CONFIRMED WORKING]. `@RPPROL` correctly resolved to $2,000,000 / 50,000,000 = 0.04$ (4%), and every value in the already-validated banded reinstatement premium formula scaled correctly by the same ratio from the earlier (broken, `ROL=0.05`-based) test: - Year 106 (full \$50M band): $50,000,000 \times 0.04 \times 0.5 = 1,000,000$ [OK] - Year 125 (partial): $9,600,176.25 \times 0.02 = 192,003.53$ [OK] - Year 8845: $11,777,100 \times 0.02 = 235,542$ [OK] - Year 7243 (both bands fully used): $(50M \times 0.02) + (50M \times 0.04) = 3,000,000$ [OK] - All 7 zero-loss years still correctly \$0 [OK]

This confirms the fix mechanism is fully correct — the lookup now reaches the intended value, and the rest of the already-validated formula processes it correctly with no side effects.

Still outstanding, unchanged from before: this remains a **mechanically-proven, business-unvalidated** fix, same caveat as the CATBOND cap in Phase 3. We still don't know, and cannot determine from code or data alone, whether LayerPremium on the container row is genuinely where a real user would enter an RPP rate in practice — only that, *if* it is, this fix correctly reaches and uses it. No real RPP structure has ever existed in production data to confirm this against.

Updated open items from Phase 4

- Same as above, plus: **LayerPremium = \$2,000,000** on the test container layer (2807) is itself test data, not a real value — revert or note clearly if this layer/instrument is retained for any other purpose.
- **Recommendation for handoff:** both Phase 3 (CATBOND cap) and Phase 4 (RPP rate lookup) fixes are now mechanically proven and ready for review — present both together to the product/actuarial owner as a paired ask, since both share the same “correctly implements the logic, but the logic’s real-world correctness needs confirmation” caveat.

Phase 5: AGGTRIGGER — Annual ILW Binary Payout (item 8)

Change 23 — Confirmed no real AGGTRIGGER layer exists

```
SELECT * FROM ARAPL_Configuration_v13.dbo.ILSTerms WHERE ReinsuranceTypeCode = 'AGGTRIGGER'
```

Empty — no real data anywhere in the database uses this type. Built an isolated synthetic test, same pattern as Phase 4’s RPP setup.

Change 24 — Setup mix-up, caught and corrected before it affected anything

New instrument/activity created: ILSInstrumentID = 339, ActivityID = 339 (“SYNTHETIC TEST - AGGTRIGGER structure”).

However, the new ILSTerms row (ILSTermsID = 2809) was found to have been inserted under ILSInstrumentID/AnalysisSID = 338 — Phase 4’s RPP test instrument — instead of the new 339. Root cause not fully pinned down (likely a placeholder value not substituted correctly during setup), but caught immediately via a verification query before running anything, and **before** any loss data was linked to the misplaced row — confirmed via ILSTermsLossSetAssociation check (see below) that Change 22’s already-confirmed RPP results were unaffected, since layer 2809 didn’t exist yet at the time that test ran.

A second issue found during cleanup: once loss data was linked (LossSetID = 900040), it appeared as **two identical duplicate rows** in ILSTermsLossSetAssociation — same pattern as an earlier duplicate-insert issue in Phase 2. Cleaned by deleting both and re-inserting once.

Corrected:

```
UPDATE ARAPL_Configuration_v13.dbo.ILSTerms
SET ILSInstrumentID = 339, AnalysisSID = 339 WHERE ILSTermsID = 2809
DELETE FROM ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation WHERE ILSTermsID = 2809
INSERT INTO ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation
(ILSTermsID, LossSetID, LMF) VALUES (2809, 900040, 1)
```

Verified clean (single association row, correct instrument/activity) before proceeding.

Layer 2809 terms: AggregateRetention = \$500,000,000, PrincipalAmount = \$100,000,000, OccurrenceLimit/OccurrenceRetention both NULL (unused by this branch), linked to real Baldwin Re loss data (LossSetID = 900040).

Change 25 — Result: [CONFIRMED WORKING]

SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_AggYLT **WHERE** ActivityID = 339 **AND** ILSTermsID = 2809

Every one of the 140 real years respects the binary payout rule exactly, with zero partial values: - AnnualLoss > 500,000,000 → NetLoss = 100,000,000 (full PrincipalAmount) in every case, from the 78 years at \$1,125,000,000 down to the smallest qualifying year (4181, \$561,973,400) - AnnualLoss ≤ 500,000,000 → NetLoss = 0 in every case, from year 125 (\$109,600,176) down to year 9907 (\$33,197,860)

This confirms the first genuinely binary (all-or-nothing) payout mechanic tested this session — distinct from every graduated/capped structure confirmed in Phases 1-4. Also confirms the Company/Industry Loss Minimum fallback (item 0.2) causes no issues when combined with this branch (no deductible flags were set on this test layer, so this specifically validates the core AGGTRIGGER mechanic in isolation, not the deductible interaction — that would need a separate combined test if desired).

Open items from Phase 5

- Test instrument 339/layer 2809 still exist — decide on cleanup/retention.
- Not yet tested: AGGTRIGGER combined with a Franchise/Reverse Franchise deductible flag (the pre-filter that gates which events count toward the annual sum) — only the core binary-payout mechanic itself has been verified so far.

Phase 6: OCCTRIGGER — Per-Occurrence Nth-Event Binary Payout (item 9)

Change 26 — Confirmed no real OCCTRIGGER layer exists; isolated test built

SELECT * FROM ARAPL_Configuration_v13.dbo.ILSTerms **WHERE** ReinsuranceTypeCode = 'OCCTRIGGER'

Empty. New isolated instrument/activity/layer built: ILSInstrumentID = 340, ActivityID = 340, ILSTermsID = 2810 (“SYNTHETIC TEST - OCCTRIGGER structure”). Verified correct instrument/activity assignment before running, given the mix-up encountered in Phase 5.

Layer 2810 terms: CompanyLossMinimum = \$10,000,000 (deliberately low, to qualify multiple real events), StartingEventNumber = 2 (should skip the 1st qualifying event, pay only the 2nd), PrincipalAmount = \$75,000,000. Linked to real Baldwin Re loss data (LossSetID = 900040).

Design rationale: deliberately targeted the one real year (7243) known from Phase 3 to contain **two** events in a single year, specifically to test whether the Nth-event logic correctly distinguishes “1st qualifying event” from “2nd qualifying event” — not just whether it triggers at all.

Change 27 — Result: [CONFIRMED WORKING], strong single-data-point proof

SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_NetYELT **WHERE** ActivityID = 340 **AND** ILSTermsID = 2810

Result: exactly one row in the entire 140-year output —

EventID=270195376, Year=7243, Loss=75,000,000

What this confirms, precisely: - Every one of the 139 single-event years was correctly skipped entirely (a lone event can only ever be “the 1st,” never “the 2nd,” so none of them should produce a payout with StartingEventNumber=2 — and none did, despite several having losses far above the \$10M threshold) - Year 7243 — the one year with two qualifying events — correctly produced **exactly one** payout, not two - The payout landed on EventID = 270195376, the later of the two events in day order (the earlier was 270195371, per Phase 3’s records) — confirming the code correctly identified and paid the **2nd** qualifying event specifically, not just any event in that year - Loss = 75,000,000 exactly matches PrincipalAmount — the binary payout, unscaled by actual event size

This is the cleanest, most unambiguous confirmation of the Nth-event mechanic obtained this session — a single precise data point (one row, one specific EventID, one exact dollar amount) rather than a broad statistical pattern, and it lands exactly where predicted by the code’s own logic.

Open items from Phase 6

- Test instrument 340/layer 2810 still exist — decide on cleanup/retention.
- Not yet tested: StartingEventNumber = 1 (default/simplest case — does it correctly pay on the *first* qualifying event when N=1, rather than requiring a 2nd event to exist at all) — only the N=2 case has been verified so far.

Phase 7: PRORATA — Ratio-Scaled Notional Tranche (item 10)

Change 28 — Market context checked first, then code traced against it

Market concept confirmed correct in the code: PRORATA represents a proportional “tranche” of a larger reinsurance layer — payout formula $(\text{NetLoss} / \text{AggregateLimit}) \times \text{PrincipalAmount}$. Traced through the code’s back-allocation step and confirmed this is exactly what it computes. The core concept is sound.

Two issues found while checking, before any test was run: 1. OccurrenceLimit/OccurrenceRetention are unconditionally overwritten to pass-through defaults at the top of this branch (SET @OccurrenceLimit = @DEFAULTLIMIT, SET @OccurrenceRetention = @DEFAULTRETENTION), silently discarding any real values a user might configure on the layer. 2. **A predicted bug:** the universal Nth-event mechanic ($\text{AggregateRetention} += (\text{EventNumber} - 1) \times \text{OccurrenceLimit}$, confirmed correct elsewhere in Phases 1-2) uses OccurrenceLimit — but issue #1 has already hardcoded that to the “unlimited” placeholder (1e15) one line earlier. Predicted: any PRORATA layer with StartingEventNumber ≥ 2 would inflate AggregateRetention to an astronomical, unreachable number, causing the layer to pay \$0 regardless of actual losses.

Change 29 — Confirmed no real PRORATA layer exists; isolated test built specifically to trigger the predicted bug

Built ILSInstrumentID = 341, ActivityID = 341, ILSTermsID = 2811 (“SYNTHETIC TEST - PRORATA structure”), with AggregateLimit = \$100,000,000, StartingEventNumber = 2 (deliberately chosen to trigger the predicted bug), linked to real Baldwin Re loss data (LossSetID = 900040).

Change 30 — Result: [CONFIRMED FINDING], unambiguous

SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_AggYLT **WHERE** ActivityID = 341 **AND** ILSTermsID = 2811

Every one of the 140 real years shows NetLoss = 0 — including 78 years at \$1,125,000,000 and dozens more in the hundreds of millions, all far exceeding the \$100,000,000 AggregateLimit. Confirmed via Debug_GRYELT that real loss data (137 events, ~\$118 billion raw sum) genuinely reached this layer, and via Debug_LayerSummary that occurrence pass-through worked correctly (GU_AAL =

GR_AAL = PostDed_AAL = Occ_AAL, all matching) — the failure is isolated specifically to the aggregate step, exactly where the predicted bug lives.

[CONFIRMED FINDING]: any PRORATA layer configured with StartingEventNumber ≥ 2 will pay \$0 regardless of actual losses, no matter how large. This is not an edge case — it's guaranteed by the code's own arithmetic every time that combination occurs, since OccurrenceLimit is unconditionally hardcoded to 1e15 for this branch one line before the Nth-event formula uses it.

Root cause, precisely: the branch's own SET @OccurrenceLimit = @DEFAULTLIMIT (intended to make occurrence terms a pass-through for this ratio-based structure) has an unintended side effect — it also breaks the *shared* Nth-event mechanic that every other branch relies on being fed a real, finite OccurrenceLimit. PRORATA is the only branch that hardcodes OccurrenceLimit to the unlimited placeholder *and* still runs the Nth-event formula afterward, making it the only branch where this interaction can occur.

No fix proposed yet. Unlike the CATBOND/RPP findings, this one likely has a low-risk, obvious-looking fix (e.g., compute the Nth-event retention offset before overwriting OccurrenceLimit, or use a fixed reference value instead of OccurrenceLimit for this specific branch's offset calculation) — but per the same discipline applied to items 6 and 7b, no fix should be written and tested until confirming with the owner whether StartingEventNumber is even a field expected to be used on PRORATA layers in practice (recall: no real PRORATA layer exists in production data at all, so — like RPP — there is nothing to validate any fix against).

Change 31 — Fix drafted, applied, and confirmed working (with a real deployment detour along the way)

Fix: inside the PRORATA branch specifically, changed:

SET @AggregateRetention = @AggregateRetention + (@EventNumber - 1) * @OccurrenceLimit
to:

SET @AggregateRetention = @AggregateRetention + (@EventNumber - 1) * @AggregateLimit
(@AggregateLimit is this branch's real, finite equivalent of "one full recovery," since @OccurrenceLimit is unconditionally hardcoded to the unlimited placeholder two lines earlier, specifically for this branch.)

Deployment took several attempts, worth documenting since it revealed something real: an early attempt at this edit was mistakenly applied to the CATXOL/RPP/CATBOND branch's nearly-identical line instead of PRORATA's — an easy mix-up, since both branches have a line of the same shape. This went undetected for several test cycles because none of the CATXOL-family layers tested so far (345, 2805, 2808) had StartingEventNumber ≥ 2, so (EventNumber - 1) = 0 made the mistake arithmetically invisible in every result checked. Caught via sp_helptext inspection and a controlled test-marker deployment check (confirming ALTER PROCEDURE itself was reaching the database correctly), then corrected: the CATXOL branch was reverted to its original, correct * @OccurrenceLimit form, and the actual fix was reapplied to the correct PRORATA line, delivered as one complete, verified file to avoid any further line-location errors.

Result: [CONFIRMED WORKING]. Verified against every one of the 140 real years, including exact arithmetic matches on partial (non-capped) cases — not just "large values became nonzero": - Year 9907 (\$33,197,860, below retention): NetLoss = 0 [OK] - Year 904 (\$193,226,500): 193,226,500 - 100,000,000 = 93,226,500 → NetLoss = 93,226,500 [OK] (exact) - Year 8845 (\$111,777,100): NetLoss = 11,777,100 [OK] (exact) - Year 4687 (\$129,476,700): NetLoss = 29,476,700 [OK] (exact) - Year 106 and every \$1,125,000,000 year: correctly capped at AggregateLimit = 100,000,000 [OK] - All 7 years below the \$100M retention: correctly 0 [OK]

This closes out item 10 (PRORATA) completely — both the bug and its fix are now confirmed, with the same "mechanically proven, business intent unconfirmed" caveat as items 6 and 7b, since PRORATA has no real production data or PDF documentation to validate the underlying business rule against.

Open items from Phase 7

- Test instrument 341/layer 2811 still exist — decide on cleanup/retention.
 - Issue #1 from Change 28 (OccurrenceLimit/OccurrenceRetention silently discarded for every PRORATA layer) — confirmed by code reading, not yet independently demonstrated with a live test.
 - PRORATA with StartingEventNumber = 1 (default/no-offset case) — not yet explicitly tested in isolation, though implicitly covered by every other item-10 test run so far (all used StartingEventNumber = 2 deliberately).
-

Phase 8: OCCILW — Occurrence-Capped, Proportionally-Scaled ILW (item 11)

Change 32 — Confirmed no real OCCILW layer exists; code check found no PRORATA-style bug

Before building a test, checked this branch's code directly (already reviewed while investigating Phase 7): **OCCILW does not reset @OccurrenceLimit to the unlimited placeholder** the way PRORATA does — its Nth-event offset line (`@AggregateRetention += (@EventNumber-1) * @OccurrenceLimit`) uses a real, finite value, consistent with the already-confirmed-correct AGGXOL/CATXOL branches. No bug predicted here; test built to confirm this branch's two combined mechanics (occurrence capping + proportional PrincipalAmount/AggregateLimit scaling) work correctly together, not to chase a specific suspected issue.

Built isolated test: ILSInstrumentID = 342, ActivityID = 342, ILSTermsID = 2812 ("SYNTHETIC TEST - OCCILW structure"). Terms: OccurrenceLimit = \$50M, OccurrenceRetention = \$100M, AggregateLimit = \$50M, AggregateRetention = 0, PrincipalAmount = \$25M (chosen to give a clean, easily-verified 0.5 scaling factor). Linked to real Baldwin Re loss data (LossSetID = 900040).

Change 33 — Result: [CONFIRMED WORKING]

SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_AggYLT **WHERE** ActivityID = 342 **AND** ILSTermsID = 2812

Verified exact arithmetic on every case, both mechanics working correctly together:

- Standard \$50M years (~133 of them): $50,000,000 \times 0.5 = 25,000,000$ [OK] - Year 125 (\$9,600,176.25): $\times 0.5 = 4,800,088.13$ [OK] exact - Year 4687 (\$29,476,700): $\times 0.5 = 14,738,350$ [OK] exact - Year 8845 (\$11,777,100): $\times 0.5 = 5,888,550$ [OK] exact - **Year 7243** (two-event year, AnnualLoss = 100,000,000): confirms the aggregate cap and proportional scaling apply in the correct order — occurrence-limited total (100M) first capped at AggregateLimit (50M), **then** scaled by 0.5 $\rightarrow$ NetLoss = 25,000,000, not $50,000,000 \times 0.5$ applied incorrectly some other way - All 7 zero-loss years: correctly 0 [OK]

No bugs found. Item 11 closes out clean — first "just verify it works" result since AGGTRIGGER (item 8) and OCCTRIGGER (item 9); no fix needed.

Open items from Phase 8

- Test instrument 342/layer 2812 still exist — decide on cleanup/retention.
-

Phase 9: ELSE Catch-All — Investigation Redirected to a Real Finding (item 14)

Change 34 — Pre-test check found something more significant than expected

Before building a synthetic test, checked for any real layer already falling into the ELSE catch-all (any ReinsuranceTypeCode not matching one of the 10 known types). **Found ReinsuranceTypeCode = 'GU'** — not a typo or accidental fallthrough, but a systematically-used type across **dozens of real ILSTerms rows**, spanning at least 3 distinct real instruments discovered this session (Baldwin Re's instrument, plus two more newly found: ILSInstrumentID = 11 — "CB-Recoletos Re DAC Series 2024-1 Class A" — and others: 30, 60, 101, 114, 129, 179, 180, 188, 267, 283-285, 296, 301-304, 316, 317, 319, 327, 331, 332).

Hypothesis considered: GU (plausibly "Gross-Up," matching the procedure's own #tmpGUYELT table naming) might be a deliberate, intentional layer type used for reporting/aggregation purposes, where the ELSE branch's plain pass-through is correct-by-design, not an oversight.

Change 35 — Systematically checked how GU layers actually receive loss data

```
SELECT t.ILSTermsID, t.ILSInstrumentID, t.AnalysisSID, a.LossSetID, a.LMF
FROM ARAPL_Configuration_v13.dbo.ILSTerms t
INNER JOIN ARAPL_Configuration_v13.dbo.ILSTermsLossSetAssociation a
ON a.ILSTermsID = t.ILSTermsID
WHERE t.ReinsuranceTypeCode = 'GU'
```

Result: empty. Not one GU layer, anywhere in the database, has a direct loss-set association.

```
SELECT ParentILSTermsID, COUNT(*) AS ChildCount
FROM ARAPL_Configuration_v13.dbo.ILSTerms
WHERE ParentILSTermsID IN (
    SELECT ILSTermsID FROM ARAPL_Configuration_v13.dbo.ILSTerms
    WHERE ReinsuranceTypeCode = 'GU')
GROUP BY ParentILSTermsID
```

Result: also empty. No GU layer has any child layers feeding it either — ruling out the second possible loss-data pathway (the "underlying layers" mechanism).

Change 36 — Confirmed directly on real, already-completed production data

Ran the real, existing ActivityID = 11 (Recoletos Re, JobStatus = 128 — a completed real analysis) with @DEBUG = 1. **Confirmed safe to do so:** the procedure's entire block that writes real results (arapl_populate_ilsresults, ILSHistorical, JobStatus update, etc.) is gated IF @DEBUG = 0 — running with @DEBUG = 1 only populates debug tables, doesn't touch or overwrite the real completed analysis.

```
SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_GUYELT WHERE ActivityID = 11 AND ILSTermsID = 62
SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_GRYELT WHERE ActivityID = 11 AND ILSTermsID = 62
SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_tmpGrLoss WHERE ActivityID = 11 AND ILSTermsID = 62
```

Result: all three empty, at every stage of the pipeline, on a real, completed production activity.

[CORRECTED] Structural pattern confirmed — but NOT confirmed as a production data gap

Important correction, caught before this went further: the investigation above (Changes 34-36) established that no GU layer has a direct loss-set association, and separately, that no Loss-SetMetaData/YELT_* data exists in *this* database for the ~25 real instruments carrying GU layers. **This does not mean the gap is real in production.** This working database was only ever populated with full loss data for a small number of test deals (primarily Baldwin Re, ActivityID = 40) — the absence of loss data for other real instruments here simply reflects that this environment was never meant to mirror full production data, not that those instruments are actually missing their real loss data somewhere in the live system.

What remains genuinely established, independent of the above correction: - **Structural pattern (confirmed via configuration, not loss data):** every GU layer has a ParentILSTerm-sID pointing to a real, correctly-typed CATBOND layer, with ParentRelationType = 3 (a value not otherwise seen in this session's testing — falls into the same "sourced/keep losses" code path as ParentRelationType = 1, so behaviorally equivalent, though its separate existence as a distinct value is itself unexplained). - **Confirmed by the person running this investigation:** GU is intended to carry real gross loss straight through with no terms applied — consistent with what the ELSE branch's code already does. - **What is NOT established:** whether real loss data for these ~25 instruments is actually missing in production, or whether it exists there and simply wasn't copied into this working environment. **This cannot be determined from this database alone.**

Revised recommendation for handoff: rather than reporting this as a confirmed production data gap, report it as: *"GU layers are structurally wired to receive loss from their parent CATBOND layers, consistent with their intended design (confirmed). This working database doesn't have loss data loaded for these instruments, so whether real production data is correctly linked for them could not be verified here — worth a direct check against the production/full environment before treating this as an issue."*

Open items from Phase 9

- **Verify against production/full environment (not this working database) whether GU layers for these ~25 real instruments have real loss-set associations and real YELT data.** This is the one check that would confirm or rule out an actual gap — not resolvable in this environment.
- The synthetic ELSE-branch test originally planned (an isolated layer with a made-up ReinsuranceTypeCode) was never built, since real GU data gave a more informative, real-world answer — the pass-through mechanism itself is already confirmed correct via this investigation, so this synthetic test is no longer needed.
- The unexplained ParentRelationType = 3 value (distinct from 1, 2, 4 seen elsewhere) — falls into the same code path as 1, so not currently causing any behavioral difference, but worth understanding why it exists as a separate value if this area is revisited.

Change 37 - Synthetic test confirming the GU/CATBOND mechanism works correctly with real loss data

Built an isolated instrument (ILSInstrumentID = 343, ActivityID = 343) mirroring the exact real-world pattern found above: a CATBOND parent (ILSTermsID = 2813, InuringSequenceNumber = 1) with a GU child (ILSTermsID = 2814, InuringSequenceNumber = 2, ParentRelationType = 3 - matching the real pattern exactly, not substituted with 1). Linked the child to real Baldwin Re loss data (LossSetID = 900040).

Result: [CONFIRMED WORKING], both halves of the mechanism:

1. GU pass-through itself - Debug_tmpGrLoss for the child (2814) shows every one of the 137 real events with Loss exactly matching raw, unmodified loss - no occurrence limit, retention, or deductible applied anywhere. Confirmed even on the split multi-event year (7243: two rows, \$873,926,400 and

\$251,073,600, summing to the real \$1,125,000,000) - proving the pass-through preserves per-event granularity correctly, not just aggregate totals.

2. Parent-child wiring (including the previously-untested ParentRelationType = 3) - Debug_GUYELT for the parent (2813) shows 137 events, AAL = 11,802,653.69 - an exact match to the AAL calculated directly from raw YELT_40 data earlier this session (\$118,026,536,852.995 / 1e4). Confirms the child's pass-through output flows into the parent's gross-up step with zero loss, and that ParentRelationType = 3 behaves identically to 1 in practice, not just by code inspection.

This confirms the entire GU/CATBOND structural pattern is mechanically sound. If real production data genuinely is missing for the ~25 real instruments using this pattern (still unverified - see open items above), the fix would be exactly as simple as it was for Baldwin Re's 344/346 in Phase 1 - just populating the missing ILSTermsLossSetAssociation rows. No deeper structural problem exists in the mechanism itself.

Item 14 fully closes out here - both the ELSE branch's correctness and the specific GU/CATBOND real-world pattern are now confirmed working, with the one remaining open question (does production actually have this data) correctly scoped as something only checkable against the real environment, not this working database.

Change 15 — Proposed fix drafted (IMPORTANT: unvalidated against product intent — see caveat below)

Explicit distinction from every fix in Phases 1-2: the Company/Industry Loss Minimum fallback and all four deductible-type fixes were each confirmed against *two independent sources* — the PDF's documented behavior, and real test data. **This fix has only one source: general CAT bond market convention** (a fully-collateralized instrument cannot lose more than its posted principal). **The PDF does not cover CAT Bond mechanics at all** — no chapter addresses this structure. So while the underlying *finding* (Change 14: modeled loss can exceed principal by 2.5x) is solidly confirmed, **the specific correction below is a proposal, not a validated fix, until confirmed by whoever owns this product's CAT bond structuring logic.**

Proposed change — inside the existing IF @ReinsuranceTypeCode = 'CATBOND' AND @OccurrenceLimit = @DEFAULTLIMIT branch, add one line so the branch's own (unmodified) shared occurrence-terms CASE expression does the capping:

```
IF @ReinsuranceTypeCode = 'CATBOND' AND @OccurrenceLimit = @DEFAULTLIMIT
BEGIN
    -- PROPOSED FIX (unvalidated - see Change 15 caveat in log): cap the occurrence
    -- limit at the bond's own principal, so a fully-collateralized bond cannot report
    -- a modeled loss exceeding its own principal. Confirmed via testing (Change 14)
    -- that without this, modeled loss can reach 2.5x PrincipalAmount. Needs sign-off
    -- from whoever owns CAT bond structuring intent before treating as a real fix.
    SET @OccurrenceLimit = @PrincipalAmount
    SET @AggReinsPrem = 0
END
ELSE
BEGIN
    ...    -- UNCHANGED - standard CATXOL/CATBOND path, already confirmed working (Phases 1-2)
END
```

Why this specific approach: reuses the existing, already-correct shared occurrence-terms CASE expression (no new logic needed) by simply correcting what @OccurrenceLimit resolves to for this one scenario. Does not touch AggregateLimit/AggregateRetention (left "as-is," matching the branch's original comment intent) or the ELSE branch at all.

Test plan (mechanical validation only — confirms the code does what we intend, not that the intent itself is correct): 1. Apply the one-line addition to the deployed procedure 2. Re-run against existing test layer 2806 (already isolated, already linked to real data) 3. Check De-

bug_0ccYELT — expect every row to now show Loss $\leq$ 250,000,000, with the previously-\$625M events reading exactly 250,000,000

Status: APPLIED and MECHANICALLY CONFIRMED.

```
SELECT TOP 10 EventID, YearID, Loss
FROM ARAPL_Analysis_V13.dbo.Debug_0ccYELT
WHERE ActivityID = 40 AND ILTermsID = 2806
ORDER BY Loss DESC
```

Result: every one of the top 10 events (previously 625,000,000) now reads **exactly 250,000,000** — the code correctly caps at PrincipalAmount, precisely as designed.

What this proves, and what it does NOT prove: - [OK] **Proves:** the SQL change is syntactically correct, deployed successfully, and produces exactly the arithmetic result intended by the fix as written. - [X] **Does NOT prove:** that capping at PrincipalAmount is actually the correct business rule for this branch. That determination rests entirely on confirmation from whoever owns CAT bond structuring intent for this product — this fix should be treated as **proposed and pending sign-off**, not production-ready, until that review happens.

Recommendation for handoff: present Change 14 (the confirmed finding, with its market-context-independent evidence — the code's own comment, the real \$625M vs \$250M discrepancy, the ~78/140-year prevalence) and Change 15 (this proposed fix, clearly labeled as a draft) together to the product/actuarial owner, and let them confirm or redirect before this goes anywhere near a real production run.

Phase 10: Drop-Down Cover (item 15)

Change 38 - Verified drafted fix against PDF's own worked example before deploying

DropDown_Cover_Fix.sql (drafted earlier in this engagement) was checked by hand against the PDF's Chapter 5 worked example (Primary: OccLimit=\$30M/OccRet=\$60M; Step-layer: OccLimit=\$30M/OccRet=\$30M; 4 events: \$40M/\$90M/\$20M/\$50M, expected results: \$0/\$30M/\$0/\$20M). Traced the drafted logic's math by hand against all 4 events and confirmed it matches exactly, assuming PrimaryReinstatement=0 (a single-shot primary, matching the PDF's simple example with no reinstatement).

Change 39 - Confirmed StepLayerSequenceNumber has never been wired end-to-end

Traced the full pipeline: StepLayerSequenceNumber exists in TSRe's source schema (tReinsurance-Treaty) but is never carried into ILTerms by ARAPL_populate_ILTerms (confirmed earlier this session), and ILTerms itself has no column for it. arapl_RunILSAnalysis_SR has no logic reading it at all. This is a three-layer gap (source data exists -> conversion never carries it -> engine has no logic), not just a missing calculation. No real production data exists anywhere to test this against.

Scope decision: added StepLayerSequenceNumber as a TEST-ONLY column on ILTerms in this working database, to test the calculation-engine logic in isolation from the separate (and larger) TSRe-conversion gap, which needs its own fix in ARAPL_populate_ILTerms and is out of scope for this session.

```
ALTER TABLE ARAPL_Configuration_v13.dbo.ILTerms ADD StepLayerSequenceNumber INT NULL
```

Linking design (reuses existing patterns rather than inventing new ones): step-layer's ParentILTermsID points to the primary's ILTermsID, with a new ParentRelationType = 5 (distinct from 1/2/4 already in use) marking it specifically as a drop-down step.

Change 40 - Engine-side wiring built and deployed

Added a new dispatch branch to arapl_RunILSAnalysis_SR, inserted before the existing AG-GXOL/QS/AQS check so none of the seven already-confirmed branches are disturbed: - ParentRelationType = 5 (this layer IS a step-layer) -> produces no standalone output; its terms are only ever used from within the primary's own processing pass - Primary layer with a linked step-layer found -> resolves the step-layer's OccurrenceRetention/OccurrenceLimit, then runs a per-event running-count calculation (SUM(TriggersPrimary) OVER (PARTITION BY YearID ORDER BY Day ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING)) to determine, for each event in chronological order, whether the primary layer's capacity (@PrimaryReinstatement, sourced from the existing @ReinstMAXID already resolved earlier in the loop for CATXOL/RPP types) has been used up yet - switching that event and everything after it in the same year onto the step-layer's terms once it has.

Deployed via complete-file ALTER PROCEDURE (same approach used for the PRORATA fix, given repeated line-location mistakes earlier this session with smaller patches).

Change 41 - Test setup, with two real mistakes caught and fixed along the way

Built isolated instrument (ILSInstrumentID = 344, ActivityID = 344), primary layer 2815 (CATXOL, OccLimit=\$30M/OccRet=\$60M, StepLayerSequenceNumber=1), step-layer 2816 (CATXOL, OccLimit=\$30M/OccRet=\$30M, ParentILSTermsID=2815, ParentRelationType=5, StepLayerSequenceNumber=2). Synthetic loss data built matching the PDF's 4 events exactly (LossSetID=900044, ActivityID=999002).

Mistake 1: loss-set association was initially linked to the step-layer (2816) instead of the primary (2815) - corrected, since only the primary should carry the association (the step-layer's terms are only ever read by reference from the primary's own processing, never run standalone).

Mistake 2: even after correcting the link, all 4 events came back exactly double their real values (e.g. \$40M input showing as \$80M). Initially suspected a synthetic-EventID collision with a real ModelCode's adjustment factor (EventIDs 1-4 resolving to ModelCode=FLOOR(EventID/1e7)=0) and rebuilt with realistic large EventIDs (999000001-999000004) - this did NOT fix it, ruling out that theory. **Real cause, confirmed directly:** the loss-set association for layer 2815 existed as two identical duplicate rows - the same category of mistake hit twice earlier this session (Phase 2's LossSetMetaData, Phase 5's ILSTermsLossSetAssociation). Deleted both and re-inserted once; doubling resolved immediately.

Change 42 - Result: [CONFIRMED WORKING], exact match to the PDF's documented example

```
SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_NetYELT WHERE ActivityID = 344 AND ILSTermsID = 2815
```

Result, matching the PDF's Chapter 5 example precisely, all four events: - Event 1 (\$40M): absent/no contract loss (below primary's \$60M retention) - CORRECT - Event 2 (\$90M): **\$30,000,000** (primary retention/limit applied; primary exhausted) - CORRECT, exact match - Event 3 (\$20M): absent/no contract loss (below step-layer's \$30M retention) - CORRECT - Event 4 (\$50M): **\$20,000,000** (step-layer retention/limit applied) - CORRECT, exact match

This confirms the entire drop-down mechanism works correctly end-to-end: the ParentRelationType=5 step-layer/primary-layer wiring, the per-event chronological running-use tracking, and the terms-switching logic all produce exactly the documented result.

Open items from Phase 10

- **This fix is validated against the PDF's documented mechanic (two independent sources: documentation + working test) - the strongest validation standard used**

this session, matching items 0.2-5. However, it has NOT been tested against real production data, since (per Change 39) no real ILSTerms row anywhere has StepLayerSequenceNumber populated - this remains purely a synthetic-data validation.

- **The upstream gap remains completely unaddressed and is a separate, larger piece of work:** ARAPL_populate_ILSTerms needs to be updated to actually carry StepLayerSequenceNumber from TSRe into ILSTerms, and a real StepLayerSequenceNumber column needs to be added to production ILSTerms (not just this test database) before any real drop-down cover could ever reach this now-working engine logic.
 - Only the simplest two-tier case (one primary, one step-layer) has been built and tested. The field's own name ("sequence number," not just a flag) suggests the original design may have anticipated more than two tiers - untested, and the drafted logic would need extending if that's ever required.
 - Test instrument 344/layers 2815-2816, synthetic loss set 900044/YELT_999002 all still exist - decide on cleanup/retention.
 - The test-only StepLayerSequenceNumber column added to ILSTerms in this working database should be removed (or clearly flagged as test-only) before this database is used for anything beyond this test, to avoid confusion with the real schema.
-

Phase 11: Multi-Child Sourcing/Inuring at One Parent (structural question, beyond original item list)

Change 43 - Question raised: does a parent with MULTIPLE children correctly combine them, with sourced vs. inuring applied independently?

Every prior multi-layer test this session (RPP container, GU/CATBOND, Drop-Down primary/step) involved exactly ONE child per parent. Combining multiple children at once, with different ParentRelationType values on each, had never been tested - only traced from the code (the underlying-layers loop uses a proper WHILE loop over every matching child, not a single lookup, and each child's own sourced-vs-inuring branch is evaluated independently when that child is processed). Given this session's repeated experience of traced-correct code turning out to have real bugs (PRORATA, RPP override), built an actual test rather than relying on the trace alone.

Change 44 - Test setup: one CATBOND parent, three CATXOL children with distinct behaviors

Built isolated instrument (ILSInstrumentID = 345, ActivityID = 345), one CATBOND parent (ILSTermSID = 2817), and three children, each with a single distinct synthetic event so each child's contribution is individually traceable in the combined result: - Child A (ILSTermSID = 2819, ParentRelationType = 1, sourced): OccLimit=\$10M/OccRet=\$0, one event Loss=\$8M -> pays \$8M, should flow up unmodified - Child B (ILSTermSID = 2822, ParentRelationType = 1, sourced): same terms, one event Loss=\$6M -> pays \$6M, should flow up unmodified - Child C (ILSTermSID = 2820, ParentRelationType = 2, inuring): OccLimit=\$5M/OccRet=\$2M, one event Loss=\$9M -> C's own recovery = min(\$9M-\$2M, \$5M) = \$5M; since inuring, what should flow up is the RESIDUAL (gross minus recovery) = \$9M - \$5M = \$4M, not C's own \$5M recovery

Setup mistakes encountered and corrected, consistent with this session's pattern: - Two child rows (2818, 2821) were created with corrupted values (ParentILSTermSID=281, ParentRelationType=71 - neither matching anything real), from a substitution error using a placeholder rather than the literal parent ID. Deleted both once confirmed genuinely broken (not a display artifact) via an explicit CAST-to-VARCHAR check. - Child B's loss-set link was initially entirely missing (0 rows), separately from Child A and Child C's links also initially missing. All three were individually verified and corrected with real LossSetIDs (900045, 900046, 900047 respectively) rather than assumed correct.

Change 45 - Result: [CONFIRMED WORKING]

SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_GUYELT **WHERE** ActivityID = 345 **AND** ILSTermsID = 28

Result: three separate rows, exactly matching prediction: - EventID 999100001 (Child A, sourced): **\$8,000,000** - exact match - EventID 999100002 (Child B, sourced): **\$6,000,000** - exact match - EventID 999100003 (Child C, inuring): **\$4,000,000** - exact match (the residual, NOT Child C's own \$5,000,000 recovery)

This confirms, for the first time with an actual test (not just code tracing): 1. The underlying-layers loop correctly pulls in and combines ALL children pointing at one parent, not just the first one found 2. Each child's own ParentRelationType (sourced vs. inuring) is evaluated and applied completely independently of its siblings - one inuring child among sourced children does not disturb or interfere with the others 3. The inuring subtraction correctly produces the residual (gross minus own recovery), consistent with how ParentRelationType=2 is documented to work elsewhere in this session's testing

Open items from Phase 11

- Test instrument 345 and all layers (2817, 2819, 2820, 2822) plus synthetic loss sets (900045-900047) still exist - decide on cleanup/retention.
- Only tested with each child having exactly one event, in the same year. Not tested: multiple children with events landing on genuinely overlapping/multi-event years, which would exercise the GROUP BY/aggregation logic at the parent more thoroughly.
- Not tested: a parent with more than 3 children, or children of mixed types (e.g., a GU child alongside a CATXOL child at the same parent) - the trace suggests this should work identically, but per this session's standard, "should work" is not the same as "confirmed."

Phase 12: Coinsurance (item 13) - Direction Confirmed, Fix Built and Tested

Change 46 - Direction confirmed by the person running this investigation

Confirmed: CoinsuranceAmount = 0.30 means the reinsurer pays 70% of the loss (the cedant's retained share is 0.30). This matches the documented formula's implication (Reinsurer's share = Layer Loss x (1 - CoinsuranceAmount)) from the earlier field-mapping work. Real-data population question (the earlier finding that Baldwin Re's CoinsuranceAmount values matched ParticipationGross suspiciously closely) explicitly deferred per instruction - this phase tests the calculation logic only.

Change 47 - Fix implemented

Added @CoinsuranceAmount (from ISNULL(CoinsuranceAmount, 0) in the terms-pull SELECT) and @CoinsuranceFactor = 1 - ISNULL(@CoinsuranceAmount, 0), declared alongside @EffectiveLossMinimum/@EffectiveLossMaximum. Applied as an additional multiplier in the final output step (the one place every layer's output passes through regardless of ReinsuranceTypeCode), alongside the existing @ParticipationNet multiplier:

```
...a.Loss * @ParticipationNet * @CoinsuranceFactor as Loss...
```

Applied to both branches of that step (the ParentRelationType=2 inuring branch and the sourced/top-layer branch).

Change 48 - Test setup and result: [CONFIRMED WORKING]

Built isolated instrument (ILSInstrumentID=346, ActivityID=346), single CATXOL layer (ILSTermSID=2823, ParticipationGross/Net=1 to isolate coinsurance's effect from participation scaling), CoinsuranceAmount=0.30, one synthetic event Loss=\$100,000.

Result: Loss = 70,000 - exact match to $100,000 \times (1 - 0.30) = 70,000$.

Open item found during this test, not yet resolved: NetLoss_xRIP came back as 100,000 - unchanged, since the fix only multiplies the Loss column, not NetLoss_xRIP, in both branches of the final output step. This wasn't a deliberate design decision during the fix - it needs an explicit answer: should coinsurance also reduce the "excluding reinstatement premium" view, or is it correct for that view to reflect the pre-coinsurance amount? Not resolved as of this entry.

Open items from Phase 12

- **Whether @CoinsuranceFactor should also apply to NetLoss_xRIP, not just Loss, in the final output step - needs an explicit decision, not yet made.**
 - Real-data population question (Baldwin Re's CoinsuranceAmount vs ParticipationGross collision) explicitly deferred this phase - still needs checking against the real TSRe source data before trusting real CoinsuranceAmount values on any actual layer.
 - Test instrument 346/layer 2823, synthetic loss set 999006/YELT_999006 still exist - decide on cleanup/retention.
-

Phase 13: Coinsurance - NetLoss_xRIP Extension and a Serious Deployment Mistake Caught

Change 49 - Decision: extend the fix to NetLoss_xRIP

Confirmed: yes, NetLoss_xRIP should also reflect coinsurance (and, incidentally, ParticipationNet - which it had never been scaled by either, in the sourced/top-layer branch specifically).

Change 50 - Deployment mistake: the actual EXECUTE line was deleted, not just edited

While extending the fix to the ELSE branch's Loss_xRIP reference, the edit that was applied accidentally REMOVED the line that executes the built query entirely:

```
IF @DEBUG = 1 PRINT @query
EXEC sp_executeSQL @query, N'@ParticipationNet FLOAT', @ParticipationNet=@ParticipationNet
```

This meant @query was still being BUILT (the SET @query = ... statements ran fine) but never RUN - it was silently overwritten by the debug block's own SET @query = 'INSERT INTO dbo.Debug_tmpGrLoss...' immediately afterward. **This was not specific to coinsurance or to this one layer** - with this line missing, NO layer of ANY type would ever populate its output table (tmpGrLoss_) for the remainder of the procedure run, since this exact code path is shared by every ReinsuranceTypeCode branch tested this entire engagement. Caught when the test came back completely blank rather than partially wrong, which was itself a signal (a genuinely wrong number would look like a real result; a total absence of output is closer to "nothing ran at all").

Fix: restored the missing IF @DEBUG = 1 PRINT @query / EXEC sp_executeSQL line, updating the parameter list to include the new @CoinsuranceFactor parameter alongside the existing @ParticipationNet:

```
IF @DEBUG = 1 PRINT @query
EXEC sp_executeSQL @query,
```

```
N'@ParticipationNet FLOAT, @CoinsuranceFactor FLOAT',  
@ParticipationNet=@ParticipationNet, @CoinsuranceFactor=@CoinsuranceFactor
```

Change 51 - Result: [CONFIRMED WORKING]

```
SELECT * FROM ARAPL_Analysis_V13.dbo.Debug_tmpGrLoss  
WHERE ActivityID = 346 AND ILSTermsID = 2823
```

Result: Loss = 70,000, NetLoss_xRIP = 70,000 - both now correctly reflect the 30% coinsurance reduction (previously NetLoss_xRIP incorrectly showed the unmodified \$100,000).

This closes out item 13 (Coinsurance) completely for the calculation-engine side - direction confirmed (reinsurer pays 1 - CoinsuranceAmount), both Loss and NetLoss_xRIP now correctly scaled, tested and exact.

Open items from Phase 13

- **Given the severity of Change 50's mistake (a missing EXECUTE line that would have silently zeroed out every layer's output, of every type, for the rest of any run), it would be prudent to re-verify a small sample of the OTHER already-confirmed items (e.g., re-run the AGGTRIGGER or OCCTRIGGER test) against the CURRENT deployed procedure, to confirm this specific mistake's window did not overlap with and silently invalidate any earlier "confirmed" result in this log.** Not yet done as of this entry.
- Real-data population question for CoinsuranceAmount (the Baldwin Re ParticipationGross collision) remains explicitly deferred, per earlier instruction - still needs checking against real TSRe source data before trusting real CoinsuranceAmount values on any actual layer.
- Test instrument 346/layer 2823 still exist - decide on cleanup/retention.

Phase 14: Quota Share Ceded % - Full Multi-Dimensional Implementation (item 12)

Change 52 - Two blocking questions resolved directly by the person running this investigation

1. Confirmed QS_Cession is a genuinely separate concept from ParticipationGross/ParticipationNet, not a duplicate - real new logic needed.
2. Confirmed cession varies by BOTH country and sub-country area (US state codes specifically) - not a flat single-level field.

Change 53 - Design evolution: from a flat column to a unified, multi-dimensional rules table

Initial design (single CededPercent column on ILSTermsRegion, keyed only by CountryID) was insufficient once state-level variation was confirmed. Further complicated when peril-level variation was also confirmed needed (Change 55 below). Investigated the real TSRe schema (tAppliesToAreaGeoXRef, uploaded by the user) and found it uses a single hierarchical GeographySID with ParentGeographySID chains (country/CRESTA/area/subarea/postal all represented as levels of ONE table, not separate columns) - a cleaner design than initially guessed, but explicitly deferred replicating that exact hierarchy-walk approach per instruction ("skip how to transfer the TSRe data ... focus on how to implement it in our SQL script").

Final design - one unified table, nullable dimension columns, NULL = wildcard:

```
CREATE TABLE ARAPL_Configuration_v13.dbo.ILSTermsCessionRules (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    ILSTermsID INT NOT NULL,
    PerilID INT NULL,
    CountryID INT NULL,
    StateCode VARCHAR(10) NULL,
    CededPercent FLOAT NOT NULL
)
```

Scoped explicitly to the three levels real event data can actually support (treaty, country, US state) - not built for finer levels (CRESTA/postal) since SummaryIndustryLoss_Stoch_Final only carries CountryID/StateCode.

Change 54 - Peril dimension added after being raised separately

Confirmed ILSTermsPeril already exists (membership-only, same pattern as ILSTermsRegion) - extended the unified ILSTermsCessionRules table to include PerilID rather than building a separate table, avoiding a combinatorial explosion of tables per dimension-combination.

Change 55 - Engine-side implementation

Terms-pull SELECT addition: @QS_Cession = ISNULL(QS_Cession, 1) - defaults to 1 (no reduction), NOT 0, so an unconfigured layer passes through unchanged rather than zeroing out - deliberately different default logic from @CoinsuranceAmount (which defaults to 0).

New block inside the AGGXOL/QS/AQS branch specifically (not other branches - cession is a quota-share-specific mechanic): resolves each event's country/state/peril via a join to SummaryIndustryLoss_Stoch_Final, then finds the single most-specific matching rule via a weighted specificity score:

```
ORDER BY
    (CASE WHEN r.StateCode IS NOT NULL THEN 4 ELSE 0 END)
+ (CASE WHEN r.PerilID IS NOT NULL THEN 2 ELSE 0 END)
+ (CASE WHEN r.CountryID IS NOT NULL THEN 1 ELSE 0 END) DESC
```

(Weights corrected mid-testing - see Change 58.)

Final output step extended (same location already modified for Coinsurance in Phase 12/13): both the inuring and sourced branches now multiply by ISNULL(c.EffectiveCession, @QS_Cession) via a LEFT JOIN to the per-event resolution table - falls back to the flat treaty default when no rule matches, and is a no-op (multiplies by 1) for any non-quota-share layer, since #tmpEventCession is simply never populated for those branches.

Also fixed while extending this block: the inuring branch's Loss/NetLoss_xRIP subtraction was missing parentheses around (b.Loss - isnull(a.Loss,0)) before the multiplier chain was applied - originally only isnull(a.Loss,0) was being multiplied, not the full subtraction. Corrected as part of this same edit.

Change 56 - Real-table protection: synthetic substitute used for testing

Explicit instruction: do not insert test data into the real, shared SummaryIndustryLoss_Stoch_Final table. Built a same-shape substitute table (TEST_SummaryIndustryLoss_Stoch_Final, EventID/CountryID/StateCode/PerilID) and temporarily repointed the one join in the deployed procedure at it for testing, clearly commented as TEST-ONLY with a revert reminder.

Change 57 - Test design and execution: 5 events, each isolating one specificity level

Built isolated instrument (ILSInstrumentID=347, ActivityID=347), one QS layer (ILSTermsID=2824, QS_Cession=0.20 treaty default), four session rules (Wind-only=0.40, US-only=0.30, US+FL=0.50, Wind+US+FL=0.60, using real PerilID/CountryID values confirmed from PerilTable/CountryTable: Wind=TC=PerilID 2, Flood=IF=PerilID 7, US=CountryID 168), and 5 synthetic events (\$100k-\$140k) in the TEST substitute table, each hitting a different specificity level (no match/peril-only/country-only/state-only/combined).

Result - first run (original weights, StateCode=4/CountryID=2/PerilID=1): - Event A (EQ/TX): 30,000 (US-only rule matched - correct, per rules as configured) - Event B (Wind/TX): 33,000 (US-only rule beat Wind-only rule, since CountryID outweighed PerilID in the original scoring) - Event C (Flood/TX): 36,000 (US-only, correct) - Event D (Flood/FL): 65,000 (US+FL state rule, correct) - Event E (Wind/FL): 84,000 (combined rule, correctly beat all single-dimension rules)

All 5 outputs matched the deployed logic's own scoring formula exactly - confirmed the lookup MECHANISM works correctly. Event B's result exposed an unconfirmed design assumption (geography outranking peril) that had never actually been confirmed with the business.

Change 58 - Priority correction confirmed and applied

Confirmed: peril-specific AND state-specific rules should both outrank a country-only rule. Scoring weights corrected: StateCode=4, PerilID=2, CountryID=1 (PerilID and CountryID weights swapped from the original). Deployed via complete-file replacement. Confirmed via direct inspection of the currently-deployed procedure text that both the corrected weights AND the revert to the real SummaryIndustryLoss_Stoch_Final table (Change 56's test substitute) landed correctly together in the same deployment.

Not yet independently re-run with the corrected weights - Event B's expected new result (44,000, Wind-only rule now winning over the 33,000 previously produced by the US-only rule) has not been directly reverified through execution as of this log entry; confirmed only by code inspection that the corrected formula is deployed. The underlying lookup mechanism itself was already proven correct across all 5 specificity cases in Change 57 - only the relative weighting of two dimensions changed.

[CONFIRMED WORKING - mechanism] / [PENDING - final priority reverification] Summary for item 12

What is fully confirmed via actual test execution: - The unified rules table correctly stores and distinguishes treaty/country/state/peril/combined rules - The most-specific-match-wins lookup correctly resolves the right rule per individual event, not per layer - Falls back correctly to the treaty-wide default when no rule matches - Correctly applied only to AGGXOL/QS/AQS layers, correctly a no-op elsewhere - Confirmed to correctly support "override just one state, leave everything else unchanged" via the QS_Cession=1 default design (reasoned through, not yet independently executed as its own dedicated test)

What is confirmed by code inspection but not yet re-executed: - The corrected peril/state-beats-country priority weighting

Open items from Phase 14

- **Re-run the 5-event test (via the TEST_ substitute table) to directly confirm Event B now returns 44,000 under the corrected weights, then revert the join back to the real table** - not yet done as of this entry.
- Real event-to-GeographySID/state translation for actual TSRe-sourced treaties remains unbuilt (explicitly deferred per instruction) - this phase only proves the calculation-engine logic works

correctly given correctly-populated input; wiring real TSRe session data into ILSTermsCessionRules is separate, later work.

- TEST_SummaryIndustryLoss_Stoch_Final (the substitute table) and test instrument 347/layer 2824 still exist - decide on cleanup/retention.
- The “override just FL only, all other states pass through unchanged” scenario was reasoned through analytically (via the QS_Cession default of 1) but not built as its own explicit, isolated test case.

Phase 15: Quota Share Ceded % - Design Redesigned Using Real AIR Geography Data (item 12, final version)

Change 59 - Real ARAPL_populate_ILSTerms source finally obtained and reviewed

The actual TSRe-to-ILSTerms population procedure (never previously uploaded this engagement, despite being referenced throughout) was reviewed directly. Confirmed, from the real source code rather than inference: - ILSTermsRegion IS populated from TSRe, via a real join through tAppliesToAreaGeoXRef -> tGeography - but the script only ever extracts the COUNTRY code, discarding any finer (state/subarea) level the source geography actually represents. - QS_Cession is explicitly hardcoded to NULL in the INSERT column list, despite CededPercent sitting one join away, unused, in the very same query that resolves country. - DeductibleType, AggregateDeductible, LayerPremium, StepLayerSequenceNumber confirmed absent from the ~55-column real INSERT list, independently confirming the earlier document-based finding. - ParentRelationType can only ever resolve to 1, 2, or 3 from this real procedure - confirming and explaining the real GU layers' ParentRelationType=3 found in Phase 9 (sourced, at a deeper inuring level than 1). ParentRelationType=4 (RPP) and =5 (Drop-Down, invented this engagement) are never produced by this procedure - any real RPP or Drop-Down structure must be entered through a separate, unseen process.

None of this invalidates any prior fix or test in this log - this is a downstream input into arapl_RunILSAnalysis_SR, not the calculation logic itself. It only sharpens several already-known population gaps from inference to code-verified fact.

Change 60 - Real AIR geography hierarchy discovered and confirmed usable

User provided a real geography table (arapl_Works_v13.dbo.temp_TblAreaExternal) with a proper level hierarchy (intLevel 1=country, 2=state, 3=county, 4=ZIP), each level carrying its own unique GeographySID. Confirmed via direct query: US (country) = GeographySID 234, Georgia (state) = GeographySID 251 - both exist as clean, standalone rows at their respective levels, not just as de-normalized labels on leaf-level rows.

Change 61 - Design iterated substantially before settling

Original Phase 14 design (ILSTermsCessionRules: PerilID/CountryID/StateCode as three flat columns) was superseded once the real GeographySID hierarchy became available - redundant with better, real source data. Went through several design iterations: 1. Considered adding GeographySID/CededPercent directly to the existing, real ILSTermsRegion table (which already holds real CountryID data from TSRe) - confirmed safe, since the existing legacy isAllRegions check (CountryID=3) remains completely unaffected by the new nullable columns. 2. Discovered a second real, pre-existing table, ILSTermsPeril (peril membership, same pattern as ILSTermsRegion) - confirmed via real data (IPID=6559, PerilID=1, ILSTermsID=2733). 3. Initially proposed a third table, ILSTermsCessionCombined, to handle “this exact peril in this exact place” cases (confirmed needed via a real business example: EQ specifically in California priced differently than EQ alone

or CA alone) - built, tested with a stray row, then found unnecessary. 4. **Final design:** researched how AIR/Verisk and the broader market actually structure this (peril-first, geography nested within peril - directly evidenced by AIRIndustry's own PerilRegion table, which is peril-first structured). Consolidated to just TWO tables: ILTermsRegion (geography-only rules) and ILTermsPeril extended with an optional GeographySID column (peril rules, optionally scoped to a specific place). ILTermsCessionCombined was dropped as unnecessary - "peril+place" is just a ILTermsPeril row with GeographySID populated, not a separate concept.

Change 62 - Final lookup design (four-level priority, two tables)

```
EffectiveCession = COALESCE(
  -- 1. Most specific: this exact peril, at this exact state (the "combined" case)
  (SELECT TOP 1 CededPercent FROM ILTermsPeril p
   WHERE p.ILTermsID=@ILTermsID AND p.PerilID=eg.PerilID
        AND p.GeographySID=eg.StateGeographySID),
  -- 2. State-wide rule, any peril
  (SELECT TOP 1 CededPercent FROM ILTermsRegion r
   WHERE r.ILTermsID=@ILTermsID AND r.GeographySID=eg.StateGeographySID
        AND r.CededPercent IS NOT NULL),
  -- 3. This peril, no specific place
  (SELECT TOP 1 CededPercent FROM ILTermsPeril p
   WHERE p.ILTermsID=@ILTermsID AND p.PerilID=eg.PerilID AND p.GeographySID IS NULL),
  -- 4. Country-wide rule, any peril
  (SELECT TOP 1 CededPercent FROM ILTermsRegion r
   WHERE r.ILTermsID=@ILTermsID AND r.GeographySID=eg.CountryGeographySID
        AND r.CededPercent IS NOT NULL)
)
```

Priority order confirmed by the person running this investigation: State beats plain Country; Peril also beats plain Country (informed by AIR's own peril-first PerilRegion table structure, confirmed as the working assumption for this design).

Two real bugs caught and fixed before testing, both the same category: the event-to-geography join (#tmpEventGeo) was initially hardcoded to strL1BriefName = 'US' for BOTH the state and country joins, regardless of an event's actual country - meaning a non-US event would incorrectly be matched against US-specific geography. Corrected to join against the event's own CountryCode (from the Stoch table) instead of a hardcoded literal, for both joins.

Change 63 - Test setup and execution: 5 events covering all 4 priority levels plus genuine fallback

Built isolated instrument (ILSInstrumentID=348, ActivityID=348), one QS layer (ILTermsID=2825, QS_Cession=0.20 treaty default). Rules: Georgia state rule (GeographySID=251, 0.50) and US country rule (GeographySID=234, 0.30) in ILTermsRegion; plain Wind rule (PerilID=2, GeographySID=NULL, 0.40) and Wind-specifically-in-Georgia rule (PerilID=2, GeographySID=251, 0.70) in ILTermsPeril. 5 synthetic events: EQ/TX, Wind/TX, EQ/GA, Wind/GA, and Flood/Japan (CountryID=49, CountryCode='JP', confirmed via real CountryTbl lookup) - the last one specifically designed to test genuine total fallback with a non-US event.

First test run came back with every event at the treaty-default rate (0.20) only - traced to the deployed #tmpEventGeo block still joining against the REAL SummaryIndustryLoss_Stoch_Final table (which has no rows for the synthetic EventIDs) rather than the TEST substitute table built for this phase, per the same real-table-protection discipline as Phase 14. Corrected the one join, re-ran.

Result: [CONFIRMED WORKING], exact match on all 5 events: - Event A (EQ/TX): 100,000 x 0.30 = 30,000 (country rule) [OK] - Event B (Wind/TX): 110,000 x 0.40 = 44,000 (plain peril rule) [OK] - Event C (EQ/GA): 120,000 x 0.50 = 60,000 (state rule, no EQ-specific peril rule exists) [OK] - Event D (Wind/GA): 130,000 x 0.70 = 91,000 (most specific: peril+place rule beat both the plain

peril rule and the plain state rule) [OK] - Event E (Flood/Japan): $140,000 \times 0.20 = 28,000$ (genuine fallback - nothing configured for this peril/country combination) [OK]

This is the strongest, most completely validated version of item 12 - built on real AIR geography data (not invented CountryID/StateCode columns), informed by real market/AIR structural convention for the priority ordering, and tested end-to-end including a genuine cross-country fallback case.

Change 64 - Test-table join reverted to the real table, confirmed via full procedure re-read

```
INNER JOIN ARAPL_IndustryLoss_V13.dbo.SummaryIndustryLoss_Stoch_Final s
ON s.EventID = g.EventID
```

Confirmed via direct inspection of the complete deployed procedure text that this revert landed correctly, and that TEST_SummaryIndustryLoss_Stoch_Final_v2 no longer appears anywhere in the deployed code.

[SUPERSEDED] Phase 14's original design

Phase 14's ILTermsCessionRules/ILTermsCessionCombined tables and their lookup logic are superseded by this phase's design and are no longer used by the deployed procedure. Recommend dropping ILTermsCessionRules (ILTermsCessionCombined was already dropped in Change 61) once confirmed nothing else references it.

Open items from Phase 15

- Test instrument 348/layer 2825, TEST_SummaryIndustryLoss_Stoch_Final_v2, and the now-unused ILTermsCessionRules table all still exist - decide on cleanup/retention.
- Real TSRe population of ILTermsRegion/ILTermsPeril's new GeographySID/CededPercent columns remains unbuilt (explicitly out of scope this engagement, per earlier instruction to focus only on the calculation-engine side) - ARAPL_populate_ILTerms would need real changes to ever populate real cession data automatically; today it hardcodes QS_Cession to NULL and discards CededPercent from its own existing join.
- The priority ordering (Peril beats plain Country) was informed by AIR's own data structure and general market convention, not a document-confirmed business rule for this specific business's treaties - worth a final confirmation from whoever structures real treaties, same caveat class as items 6/7b/10.